Project File

on

CHATBOT (IN MEDICAL LIFE)
MAJOR PROJECT –II (CSD0804[P])

Submitted in partial fulfillment of the Requirements for the award of B.E. degree

in Computer Science & Engineering

By

Name:- Anurag ,

Sourya

Roll no :-
BETN1CS22067

BETN1CS22086

Under the guidance of

Dr Nidhi Dandotiya , Assosiate Professor,CSE

ITM
UNIVERSITY
GWALIOR • MP • INDIA
"CELEBRATING DREAMS"

Department of Computer Science & Engineering

ITM University Gwalior

2026-27

CERTIFICATE

This is to certify that the Seminar entitled "Doctor Appointment Management System" is a bona fide record of the Seminar work done by ***Anurag Singh, Sourya (BETN1CS22067,BETN1CS22086)*** *under my supervision and guidance, in partial fulfillment of the requirements for the Outcome Based Education Paradigm in B.Tech VIII Sem from Department of CSE for the academic year 2026-27*

Dr Nidhi Dandotiya

(Guide)

Assosiate Professor

Dept. of CSE

Place: ITM University Gwalior

Abstract

Medical chatbots are artificial intelligence–based systems designed to assist patients and healthcare providers through automated conversations. These chatbots use technologies such as Natural Language Processing (NLP), Machine Learning (ML), and cloud computing to understand user queries and provide appropriate medical guidance. They are widely used for symptom checking, appointment scheduling, medication reminders, mental health support, and patient data collection.

Applications like Ada Health and Wysa demonstrate how AI-driven chatbots improve healthcare accessibility and efficiency. By offering 24/7 availability, reducing hospital workload, and enhancing patient engagement, medical chatbots contribute significantly to modern digital healthcare systems.

Technologies Used

- Django
- Python
- HTML
- CSS
- JavaScript

Table Of Contents

ABSTRACT 1

CHAPTER 1

INTRODUCTION…….5

1.1 Project Plan…….6

1.1.1 About the project..……6

1.1.2 Purpose and scope……7

1.2 Front End……8

1.2.1 Introduction Of HTML…….8

1.2.2 Introduction of CSS…….12

1.2.3 Introduction of JavaScript……...15

1.3 Back End……...18

1.3.1 Python…….18

1.3.2 Django…….22

CHAPETR 2

SOFTWARE DEVELOPMENT LIFE CYCLE……25

2.1 Requirement Analysis Phase….....26

2.1.1 System Requirement Specification.....27

2.1.2 Hardware and Software Requirement.....28

2.1.3 Functional Requirement 28

2.1.4 Non-Functional Requirement 29

2.1.5 Feasibility Study..... 31

2.1.6 Technical Feasibility 32

2.1.7 Economic Feasibility..... 32

2.1.8 Behavioural Feasibility 33

CHAPETR 3

DIAGRAMS…….34

1. Data Flow Diagram……...35
2. Sequence Diagram…….....37
3. Component Diagram……38
4. Use Case Diagram…….....39

CHAPTER 4

CODING……..40

4.1 PYTHON Code……..….....41

4.2 HTML Code…….51

CHAPTER 5

TESTING…….....55

5.1 Unit Testing……...56

5.2 Integration Testing……....57

5.3 System Testing……,.58

5.4 Functional Testing……....59

CHAPETR 6

SNAPSHOTS…….60

CHAPETR 7

FUTURE ENHANCEMENT……....67

CONCLUSION…….....68

REFERENCES….....69

CHAPTER 1

INTRODUCTION

Introduction

1.1 PROJECT PLAN

1.1.1 ABOUT THE PROJECT

In our daily life we everyday see we face a lot of problems. Disease is one of the most common issues for a person's life. If anybody is ill and wants to visit a doctor for checkup, he or she needs to visit the hospital and wait until the doctor is available. The patient also waits in a queue while getting an appointment and in illness he or she has to face these problems also. If the doctor somehow cancels the appointment due to some emergency reasons then the patient is not able to know about the cancellation of the appointment unless or until he or she visits the hospital.

So, it's necessary to get a consultation with Doctors whenever we get affected with various diseases. As we know that the internet is now available for everyone therefore anyone can use the online appointment system to overcome such problems and inconvenience for the patients.

The core vision of this project is to create a doctor patient handling management system that will help patients to book doctor appointments and fulfill their prospects. In this appointment system doctors are allowed to manage their booking slots online, patients can make their appointment to book empty slots too.

This is the system of reservation for counseling by patients name. This appointment system manages different kinds of doctors at a single time and also patients can choose their expected doctor for booking. After booking the slot also acknowledge to the doctor as well as patient about the booking slots.

1.1.2 PURPOSE AND SCOPE

Our main objective is helping people to search for a respective doctor and get an appointment. We are using the Django framework of python for creating a web application for creating different modules in the application. We also use HTML, CSS for creating a user experience UI. In this project a doctor gets an appointment request, accesses an appointment request, and also sees the patient's profile and also the doctor can give an appointment. On the other side a patient can view the doctors list , a patient can easily take doctors appointments, also checks at the same time whether the doctor is available or not.

We want to build an application with perfection and also collecting requirements is a must. It also provides a clearer idea of people’s needs and also explains how much we are going to cover in this phase of this project. Our motivation is if we get an appointment in a very short period of time that can be more precious for us. We have planned to implement a web based doctor appointment system by covering all of these features.

1.2 Front End

1.2.1 Introduction of HTML

HTML is an acronym which stands for **Hyper Text Markup Language** which is used for creating web pages and web applications. Let's see what is meant by Hypertext Markup Language, and Web page.

Hyper Text:

HyperText simply means "Text within Text." A text has a link within it, is a hypertext. Whenever you click on a link which brings you to a new webpage, you have clicked on a hypertext. HyperText is a way to link two or more web pages (HTML documents) with each other.

Markup language:

A markup language is a computer language that is used to apply layout and formatting conventions to a text document. Markup language makes text more interactive and dynamic. It can turn text into images, tables, links, etc.

Web Page:

A web page is a document which is commonly written in HTML and translated by a web browser. A web page can be identified by entering an URL. A Web page can be of the static

or dynamic type. **With the help of HTML only, we can create static web pages**.

Hence, HTML is a markup language which is used for creating attractive web pages with the help of styling, and which looks in a nice format on a web browser. An HTML document is made of many HTML tags and each HTML tag contains different content.

Brief History of HTML:

In the late 1980's , a physicist, Tim Berners-Lee who was a contractor at CERN, proposed a system for CERN researchers. In 1989, he wrote a memo proposing an internet based hypertext system.

Tim Berners-Lee is known as the father of HTML. The first available description of HTML was a document called "HTML Tags" proposed by Tim in late 1991. The latest version of HTML is HTML5, which we will learn later in this tutorial.

HTML Versions:

Since the time HTML was invented there are lots of HTML versions in market, the brief introduction about the HTML version is given below:

HTML 1.0: The first version of HTML was 1.0, which was the barebones version of HTML language, and it was released in1991.

HTML 2.0: This was the next version which was released in 1995, and it was standard language version for website design. HTML 2.0 was able to support extra features such as form-based file upload, form elements such as text box, option button, etc.

HTML 3.2: HTML 3.2 version was published by W3C in early 1997. This version was capable of creating tables and providing support for extra options for form elements. It can also support a web page with complex mathematical equations. It became an official standard for any browser till January 1997. Today it is practically supported by most of the browsers.

HTML 4.01: HTML 4.01 version was released on December 1999, and it is a very stable version of HTML language. This version is the current official standard, and it provides added support for stylesheets (CSS) and scripting ability for various multimedia elements.

HTML5 : HTML5 is the newest version of HyperText Markup language. The first draft of this version was announced in January 2008. There are two major organizations one is W3C (World Wide Web Consortium), and another one is WHATWG(Web Hypertext Application Technology Working

Group) which are involved in the development of HTML 5 version, and still, it is under development.

Features of HTML

1) It is a very **easy and simple language**. It can be easily understood and modified.

2) It is very easy to make an **effective presentation** with HTML because it has a lot of formatting tags.

3) It is a **markup language**, so it provides a flexible way to design web pages along with the text.

4) It facilitates programmers to add a **link** on the web pages (by html anchor tag), so it enhances the interest of browsing of the user.

5) It is **platform-independent** because it can be displayed on any platform like Windows, Linux, and Macintosh, etc.

6) It facilitates the programmer to add **Graphics, Videos, and Sound** to the web pages which makes it more attractive and interactive.

7) HTML is a case-insensitive language, which means we can use tags either in lower-case or upper-case.

1.2.2 Introduction of CSS

CSS stands for Cascading Style Sheets. It is a style sheet language which is used to describe the look and formatting of a document written in markup language. It provides an additional feature to HTML. It is generally used with HTML to change the style of web pages and user interfaces. It can also be used with any kind of XML documents including plain XML, SVG and XUL.

CSS is used along with HTML and JavaScript in most websites to create user interfaces for web applications and user interfaces for many mobile applications.

- **What does CSS do**

You can add new looks to your old HTML documents.

You can completely change the look of your website with only a few changes in CSS code.

- **Why use CSS**

These are the three major benefits of CSS:

1) Solves a big problem

Before CSS, tags like font, color, background style, element alignments, border and size had to be repeated on every web page. This was a very long process. For example: If you are developing a large website where fonts and color information

are added on every single page, it will be become a long and expensive process. CSS was created to solve this problem. It was a W3C recommendation.

2) Saves a lot of time

CSS style definitions are saved in external CSS files so it is possible to change the entire website by changing just one file.

3) Provide more attributes

CSS provides more detailed attributes than plain HTML to define the look and feel of the website.

- **Advantages of CSS**

CSS saves time − You can write CSS once and then reuse same sheet in multiple HTML pages. You can define a style for each HTML element and apply it to as many Web pages as you want.

Pages load faster − If you are using CSS, you do not need to write HTML tag attributes every time. Just write one CSS rule of a tag and apply it to all the occurrences of that tag. So less code means faster download times.

Easy maintenance − To make a global change, simply change the style, and all elements in all the web pages will be updated automatically.

Superior styles to HTML − CSS has a much wider array of attributes than HTML, so you can give a far better look to your HTML page in comparison to HTML attributes.

Multiple Device Compatibility − Style sheets allow content to be optimized for more than one type of device. By using the same HTML document, different versions of a website can be presented for handheld devices such as PDAs and cell phones or for printing.

Global web standards − Now HTML attributes are being deprecated and it is being recommended to use CSS. So its a good idea to start using CSS in all the HTML pages to make them compatible to future browsers.

➢ CSS Versions

Cascading Style Sheets level 1 (CSS1) came out of W3C as a recommendation in December 1996. This version describes the CSS language as well as a simple visual formatting model for all the HTML tags.

CSS2 became a W3C recommendation in May 1998 and builds on CSS1. This version adds support for media-specific style sheets e.g. printers and aural devices, downloadable fonts, element positioning and tables.

1.2.3 Introduction of JavaScript

JavaScript is a light-weight object-oriented programming language which is used by several websites for scripting the webpages. It is an interpreted, full-fledged programming language that enables dynamic interactivity on websites when applied to an HTML document. It was introduced in the year 1995 for adding programs to the webpages in the Netscape Navigator browser. Since then, it has been adopted by all other graphical web browsers. With JavaScript, users can build modern web applications to interact directly without reloading the page every time. The traditional website uses js to provide several forms of interactivity and simplicity.

Although, JavaScript has no connectivity with Java programming language. The name was suggested and provided in the times when Java was gaining popularity in the market. In addition to web browsers, databases such as CouchDB and MongoDB uses JavaScript as their scripting and query language.

- **Features of JavaScript**

There are following features of JavaScript:

- All popular web browsers support JavaScript as they provide built-in execution environments.
- JavaScript follows the syntax and structure of the C programming language. Thus, it is a structured programming language.

- JavaScript is a weakly typed language, where certain types are implicitly cast (depending on the operation).
- JavaScript is an object-oriented programming language that uses prototypes rather than using classes for inheritance.
- It is a light-weighted and interpreted language.
- It is a case-sensitive language.
- JavaScript is supportable in several operating systems including, Windows, macOS, etc.
- It provides good control to the users over the web browsers.

➢ History of JavaScript

In 1993, **Mosaic**, the first popular web browser, came into existence. In the **year 1994**, **Netscape** was founded by **Marc Andreessen**. He realized that the web needed to become more dynamic. Thus, a 'glue language' was believed to be provided to HTML to make web designing easy for designers and part-time programmers. Consequently, in 1995, the company recruited **Brendan Eich** intending to implement and embed Scheme programming language to the browser. But, before Brendan could start, the company merged with **Sun Microsystems** for adding Java into its Navigator so that it could compete with Microsoft over the web technologies and platforms. Now, two languages were there: Java and the scripting language. Further, Netscape decided to give a similar

name to the scripting language as Java's. It led to 'Javascript'. Finally, in May 1995, Marc Andreessen coined the first code of Javascript named '**Mocha**'. Later, the marketing team replaced the name with '**LiveScript**'. But, due to trademark reasons and certain other reasons, in December 1995, the language was finally renamed to 'JavaScript'. From then, JavaScript came into existence.

➢ Application of JavaScript

- JavaScript is used to create interactive websites. It is mainly used for:
- Client-side validation,
- Dynamic drop-down menus,
- Displaying date and time,
- Displaying pop-up windows and dialog boxes (like an alert dialog box, confirm dialog box and prompt dialog box),
- Displaying clocks etc.

1.3. Back End

1.3.1 Introduction of Python

Python tutorial provides basic and advanced concepts of Python. Our Python tutorial is designed for beginners and professionals.

Python is a simple, general purpose, high level, and object-oriented programming language.

Python is an interpreted scripting language also. Guido Van Rossum is known as the founder of Python programming.

- **What is Python**

Python is a general purpose, dynamic, high-level, and interpreted programming language. It supports Object Oriented programming approach to develop applications. It is simple and easy to learn and provides lots of high-level data structures.

Python is easy to learn yet powerful and versatile scripting language, which makes it attractive for Application Development.

Python's syntax and dynamic typing with its interpreted nature make it an ideal language for scripting and rapid application development.

Python supports multiple programming pattern, including object-oriented, imperative, and functional or procedural programming styles.

Python is not intended to work in a particular area, such as web programming. That is why it is known as multipurpose programming language because it can be used with web, enterprise, 3D CAD, etc.

We don't need to use data types to declare variable because it is dynamically typed so we can write a=10 to assign an integer value in an integer variable.

Python makes the development and debugging fast because there is no compilation step included in Python development, and edit-test-debug cycle is very fast.

➢ Python 2 vs. Python 3

In most of the programming languages, whenever a new version releases, it supports the features and syntax of the existing version of the language, therefore, it is easier for the projects to switch in the newer version. However, in the case of Python, the two versions Python 2 and Python 3 are very much different from each other.

A list of differences between Python 2 and Python 3 are given below:

Python 2 uses **print** as a statement and used as print "something" to print some string on the console. On the other hand, Python 3 uses **print** as a function and used as print("something") to print something on the console.

Python 2 uses the function raw_input() to accept the user's input. It returns the string representing the value, which is typed by the user. To convert it into the integer, we need to use the int() function in Python. On the other hand, Python 3 uses input() function which automatically interpreted the type of input entered by the user. However, we can cast this value to any type by using primitive functions (int(), str(), etc.).

In Python 2, the implicit string type is ASCII, whereas, in Python 3, the implicit string type is Unicode.

Python 3 doesn't contain the xrange() function of Python 2. The xrange() is the variant of range() function which returns a xrange object that works similar to Java iterator. The range() returns a list for example the function range(0,3) contains 0, 1, 2.

There is also a small change made in Exception handling in Python 3. It defines a keyword **as** which is necessary to be used. We will discuss it in Exception handling section of Python programming tutorial.

➢ Python History

Python was invented by **Guido van Rossum** in 1991 at CWI in Netherland. The idea of Python programming language has

taken from the ABC programming language or we can say that ABC is a predecessor of Python language.

There is also a fact behind the choosing name Python. Guido van Rossum was a fan of the popular BBC comedy show of that time, **"Monty Python's Flying Circus"**. So he decided to pick the name **Python** for his newly created programming language.

Python has the vast community across the world and releases its version within the short period.

➢ **Where is Python used?**

Python is a general-purpose, popular programming language and it is used in almost every technical field. The various areas of Python use are given below.

- Data Science
- Date Mining
- Desktop Applications
- Console-based Applications
- Mobile Applications
- Software Development
- Artificial Intelligence
- Web Applications
- Enterprise Applications
- 3D CAD Applications
- Machine Learning

1.3.2 Django

Django is a web application framework written in Python programming language. It is based on MVT (Model View Template) design pattern. The Django is very demanding due to its rapid development feature. It takes less time to build application after collecting client requirement.

This framework uses a famous tag line:

The web framework for perfectionists with deadlines.

By using Django, we can build web applications in very less time. Django is designed in such a manner that it handles much of configure things automatically, so we can focus on application development only.

➢ **History**

Django was design and developed by Lawrence journal world in 2003 and publicly released under BSD license in July 2005. Currently, DSF (Django Software Foundation) maintains its development and release cycle.

Django was released on 21, July 2005. Its current stable version is 2.0.3 which was released on 6 March, 2018.

➢ **Popularity**

Django is widely accepted and used by various well-known sites such as:

- Instagram

- Mozilla
- Disqus
- Pinterest
- Bitbucket
- The Washington Times

➢ **Features of Django**

- Rapid Development
- Secure
- Scalable
- Fully loaded
- Versatile
- Open Source
- Vast and Supported Community

➢ **Rapid Development**

Django was designed with the intention to make a framework which takes less time to build web application. The project implementation phase is a very time taken but Django creates it rapidly.

➢ **Secure**

Django takes security seriously and helps developers to avoid many common security mistakes, such as SQL injection, cross-site scripting, cross-site request forgery etc. Its user

authentication system provides a secure way to manage user accounts and passwords.

- **Scalable**

Django is scalable in nature and has ability to quickly and flexibly switch from small to large scale application project.

- **Fully loaded**

Django includes various helping task modules and libraries which can be used to handle common Web development tasks. Django takes care of user authentication, content administration, site maps, RSS feeds etc.

- **Versatile**

Django is versatile in nature which allows it to build applications for different-different domains. Now a days, Companies are using Django to build various types of applications like: content management systems, social networks sites or scientific computing platforms etc.

- **Open Source**

Django is an open source web application framework. It is publicly available without cost. It can be downloaded with source code from the public repository. Open source reduces the total cost of the application development.

CHAPTER 2

SOFTWARE DEVELOPMENT LIFE CYCLE

2.1 REQUIREMENT ANALYSIS PHASE

The Requirements Analysis Phase begins when the previous phase objectives have been achieved. Documentation related to user requirements from the Concept Development Phase and the Planning Phase shall be used as the basis for further user needs analysis and the development of detailed requirements. Multiple-release projects require only one iteration of the Requirements Analysis Phase, which should involve requirements definition for all planned releases.

The objective of this phase is to define in more detail the system inputs, processes, outputs and interfaces. At the end of this phase the system's processes will be defined at the functional level, meaning the functions to be performed will be known, but not necessarily how they will be performed. Unless specifically constrained by the Project Charter, Requirements Analysis should not consider the computer programs, files and data streams. Requirements Analysis will identify and consider the risks related to how the technology will be integrated into the standard operating procedures. Requirements Analysis will collect the functional and system requirements of the business process, the user requirements and the operational requirements (e.g., when operational what is necessary to keep the system up and running).

2.1.1 System Requirement Specification

Software requirements specification establishes the basis for an agreement between customers and contractors or suppliers on how the software product should function (in a market-driven project, these roles may be played by the marketing and development divisions). Software requirements specification is a rigorous assessment of requirements before the more specific system design stages, and its goal is to reduce later redesign. It should also provide a realistic basis for estimating product costs, risks, and schedules. Used appropriately, software requirements specifications can help prevent software project failure.

The software requirements specification document lists sufficient and necessary requirements for the project development. To derive the requirements, the developer needs to have clear and thorough understanding of the products under development. This is achieved through detailed and continuous communications with the project team and customer throughout the software development process.

Purpose

The purpose of this document is to give a detailed description of the requirements for the “Amazing Lunch Indicator” (ALI) software. It will illustrate the purpose and complete declaration for the development of system. It will

also explain system constraints, interface and interactions with other external applications. This document is primarily intended to be proposed to a customer for its approval and a reference for developing the first version of the system for the development team.

2.1.2 Hardware and Software Requirement

1. **Hardware Specification**

    - RAM 4 GB
    - Windows 10

2. **Software Requirements:**

    - Python
    - PyCharm
    - Browser to Test

2.1.3 Functional Requirement

In Software engineering and systems engineering, a functional requirement defines a function of a system or its component. A function is described as a set of inputs, the behaviour, and outputs. Functional requirements may be

calculations, technical details, data manipulation and processing and other specific functionality that define what a system is supposed to accomplish. Behavioural requirements describing all the cases where the system uses the functional requirements are captured in use cases. Functional requirements are supported by non-functional requirements which impose constraints on the design or implementation.

As defined in requirements engineering, functional requirements specify particular results of a system. This should be contrasted with non-functional requirements which specify overall characteristics such as cost and reliability. Functional requirements drive the application architecture of a system, while non-functional requirements drive the technical architecture of a system.

2.1.4 Non-Functional Requirement

In systems engineering and requirements engineering, a non-functional requirement is a requirement that specifies criteria that can be used to judge the operation of a system, rather than specific behaviours.

They are contrasted with functional requirements that define specific behaviour or functions. The plan for implementing functional requirements is detailed in the system design. The plan for implementing non-functional requirements is detailed in the system architecture, because they are usually

Architecturally Significant Requirements.

Broadly, functional requirements define what a system is supposed to do and non-functional requirements define how a system is supposed to be. Functional requirements are usually in the form of, an individual action or part of the system, perhaps explicitly in the sense of a mathematical function, a black box description input, output, process and control functional model or IPO Model. In contrast, non-functional requirements are in the form of, an overall property of the system as a whole or of a particular aspect and not a specific function. The system's overall properties commonly mark the difference between whether the development project has succeeded or failed.

Non-functional requirements are often called "quality attributes" of a system. Other terms for non-functional requirements are "qualities", "quality goals", "quality of service requirements", "constraints" and "non-behavioural requirements". Informally these are sometimes called the "ileitis", from attributes like stability and portability. Qualities—that is non-functional requirements—can be

divided into two main categories:

1. Execution qualities, such as safety, security and usability, which are observable during operation.
2. Evolution qualities, such as testability, maintainability, extensibility and scalability, which are embodied in the static structure of the system.

2.1.5 Feasibility Study

Feasibility study is made to see if the project on completion will serve the purpose of the organization for the amount of work, effort and the time that spend on it. Feasibility study lets the developer foresee the future of the project and the usefulness.

A feasibility study of a system proposal is according to its workability, which is the impact on the organization, ability to meet their user needs and effective use of resources. Thus when a new application is proposed it normally goes through a feasibility study before it is approved for development. Following are its feature

2.1.6 TECHNICAL FEASIBILITY

The system must be evaluated from the technical point of view first. The assessment of this feasibility must be based on an outline design of the system requirement in the terms of input, output, programs and procedures.

Having identified an outline system, the investigation must go on to suggest the type of equipment, required method developing the system, of running the system once it has been designed.

Technical issues raised during the investigation are:

- Does the existing technology sufficient for the suggested one?
- Can the system expand if developed?

2.1.7 ECONOMIC FEASIBILITY

The developing system must be justified by cost and benefit. Criteria to ensure that effort is concentrated on project, which will give best, return at the earliest.

One of the factors, which affect the development of a new

system, is the cost it would require.

The following are some of the important financial questions asked during preliminary investigation:

- The costs conduct a full system investigation.
- The cost of the hardware and software.
- The benefits in the form of reduced costs or fewer costly errors.

Since the system is developed as part of project work, there is no manual cost to spend for the proposed system. Also all the resources are already available, it give an indication of the system is economically possible for development

2.1.8 BEHAVIORAL FEASIBILITY

This includes the following questions:

- Is there sufficient support for the users?
- Will the proposed system cause harm?

The project would be beneficial because it satisfies the objectives when developed and installed. All behavioural aspects are considered carefully and conclude that the project is behaviourally feasible.

CHAPTER 3

DIAGRAMS

Implementation or Architecture Diagrams

1. **Data Flow Diagram (DFD)**

A data flow diagram is a graphical representation that depicts information flow and the transforms that are applied as data move from input to output. The basic form of a data flow diagram, also known as a data flow graph or a bubble chart, The data flow diagram may be used to represent a system or software at any level of abstraction. As information moves through software, it is modified by a series of transformations.

A data flow diagram is a graphical representation that depicts information flow and the transforms that are applied as data move from input to output. The basic form of a data flow diagram, also known as a data flow graph or a bubble chart. DFD is an abstract description of the system. The data flow diagram may be used to represent a system or software at any level of abstraction. DFDs may be partitioned into levels that represent increasing information flow and functional detail. Therefore, the DFD provides a mechanism for functional modeling as well as information flow modeling.

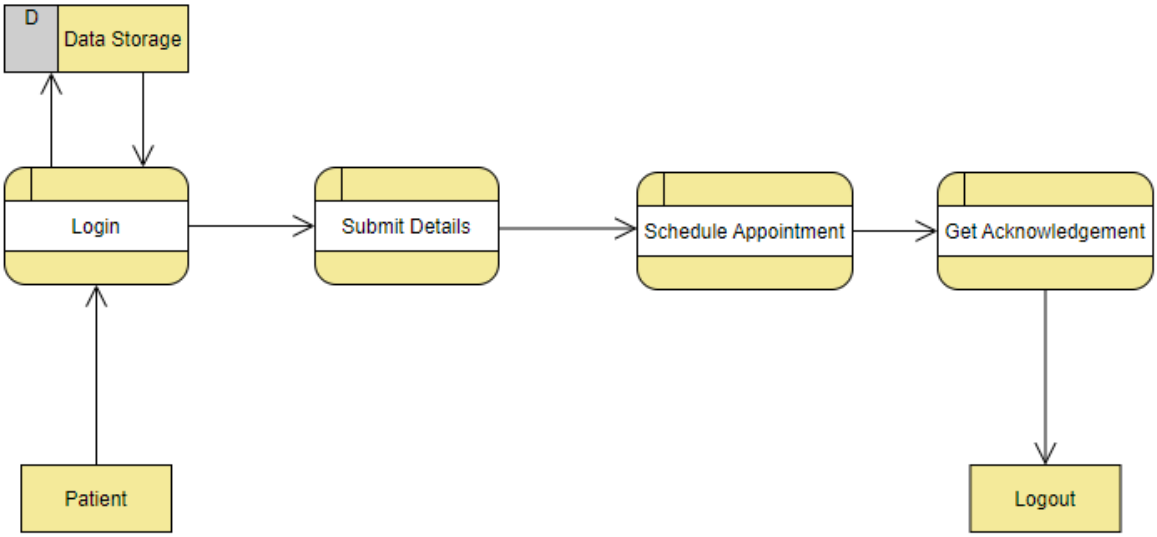
DFDs are very useful in understanding a system and can be effectively used during analysis. DFDs can be

hierarchically organized, which helps in progressively partitioning and analyzing large systems. Such DFDs are called leveled DFDs.

.

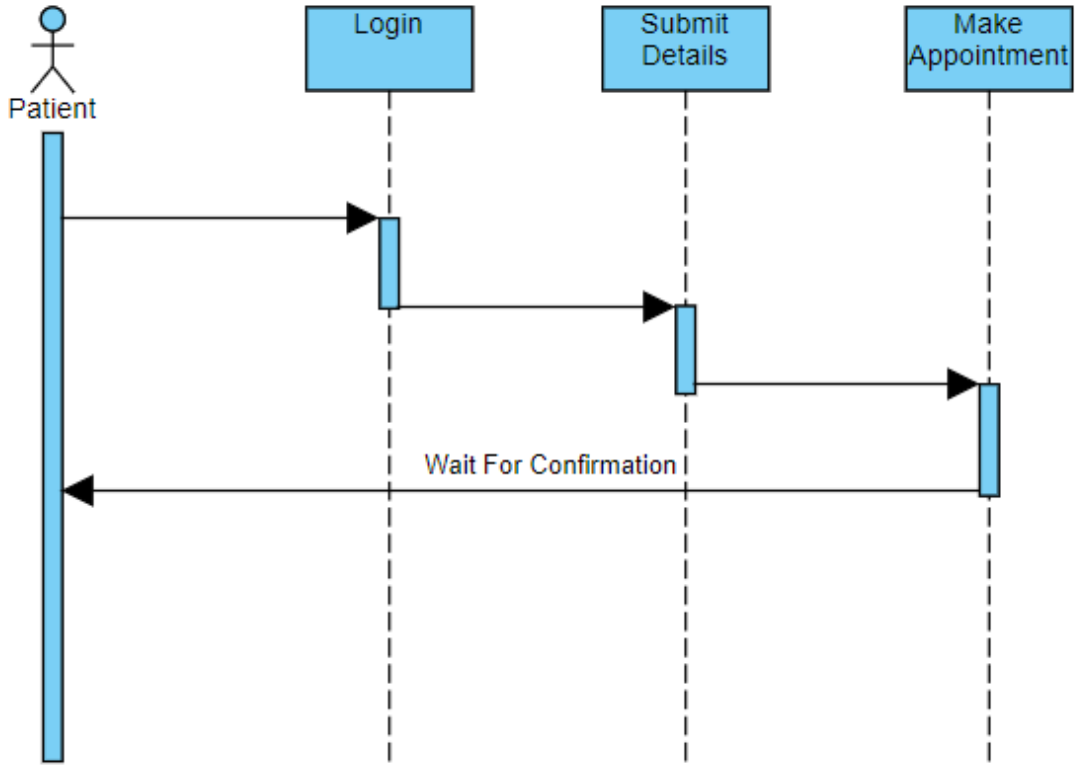
Level-0 DFD

Level-1 DFD



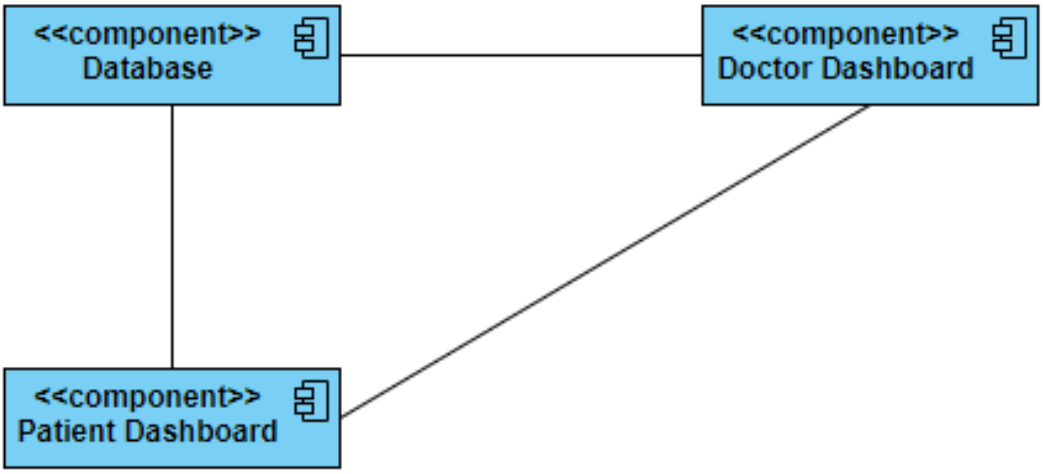
2. Sequence Diagram

A **sequence diagram** simply depicts interaction between objects in a **sequential** order i.e. the order in which these interactions take place. We can also use the terms event **diagrams** or event scenarios to refer to a **sequence diagram**. **Sequence diagrams** describe how and in what order the objects in a system function.



3. Component Diagram

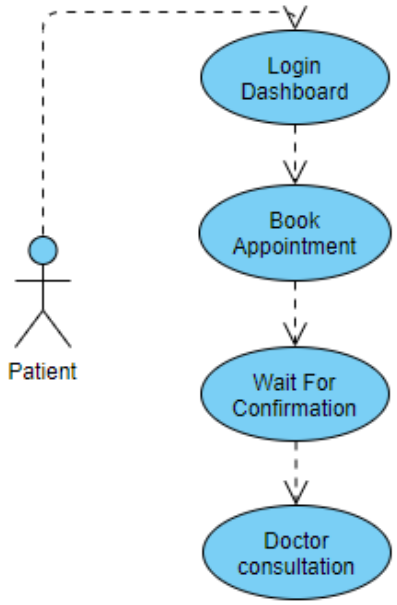
A component diagram, also known as a UML component diagram, describes the organization and wiring of the physical components in a system. Component diagrams are often drawn to help model implementation details and double-check that every aspect of the system's required functions is covered by planned development. A component diagram allows verification that a system's required functionality is acceptable. These diagrams are also used as a communication tool between the developer and stakeholders of the system. Programmers and developers use the diagrams to formalize a roadmap for the implementation, allowing for better decision-making about task assignment or needed skill improvements.



4. Use Case Diagram

To model a system, the most important aspect is to capture the dynamic behavior. Dynamic behavior means the behavior of the system when it is running/operating.

Only static behavior is not sufficient to model a system rather dynamic behavior is more important than static behavior. In UML, there are five diagrams available to model the dynamic nature and use case diagram is one of them. Now as we have to discuss that the use case diagram is dynamic in nature, there should be some internal or external factors for making the interaction. These internal and external agents are known as actors. Use case diagrams consists of actors, use cases and their relationships. The diagram is used to model the system/subsystem of an application. A single use case diagram captures a particular functionality of a system. Hence to model the entire system, a number of use case diagrams are used.



CHAPTER 4

CODING

1. Python Code

```
from django.contrib.auth import authenticate, login
from django.shortcuts import render, redirect
from django.conf import settings
import pickle
from django.core.mail import send_mail


def index(request):
    return render(request, "index.html")

def home(request):
    return render(request, "home.html")

def about(request):
    return render(request, "about.html")

def doctors(request):
    return render(request, "doctors.html")

def departments(request):
    return render(request, "departments.html")

def doctorlogin(request):

    registereddoctors = {}

    if request.method == 'POST':

        doctorusername = request.POST.get('doctorusername')
        doctorpassword = request.POST.get('doctorpassword')

        try:
            pickle_in = open("doctors.pickle", "rb")
            pickledata = pickle.load(pickle_in)
            registereddoctors = pickledata.get('registereddoctors')

        except:
            pass


        #print(doctorusername)
        #print(doctorpassword)

        canDoctorLogin = False


        if len(registereddoctors)>0:
            for i in registereddoctors:
                #print(i)
                print(registereddoctors.get(i).get("password"))
```

```
                if i == doctorusername and
registereddoctors.get(i).get("password") == doctorpassword:
                    print("Login Succesful.")
                    request.session["doctorusername"] = doctorusername

                    canDoctorLogin = True
                    break
                else:
                    print("please check your password")

            if canDoctorLogin:
                return redirect('doctordashboard')
            else:
                print("please create account first.")

        else:
            print("please create account first")





    return render(request, "doctorlogin.html")

def doctorsignup(request):

    registereddoctors = {}

    if request.method == 'POST':

        newdoctorusername = request.POST.get('newdoctorusername')
        newdoctorpassword = request.POST.get('newdoctorpassword')
        newdoctorconfirmpassword =
request.POST.get('newdoctorconfirmpassword')

        try:
            pickle_in = open("doctors.pickle", "rb")
            pickledata = pickle.load(pickle_in)
            registereddoctors = pickledata.get('registereddoctors')

        except:
            pass



        canCreateNewDoctor = True
        if len(registereddoctors)>0:
            for i in registereddoctors:
                if i == newdoctorusername:
                    print("doctor with this username already exist.")
                    canCreateNewDoctor = False

        #time_slots = {"slot1": "10 am to 11 am", "slot2": "11 am to 12 am",
        #"slot3": "1 pm to 2 pm", "slot4": "5 pm to 6 pm", "slot5": "6 pm to
7 pm", "slot6": "7 pm to 8 pm"}
```

```
        booked = []

        if canCreateNewDoctor:

            registereddoctors[newdoctorusername] =
{"username":newdoctorusername, "password":newdoctorpassword,
"booked_slots":[]}
            request.session["doctorusername"] = newdoctorusername
            send_mail(
                'Thanks to be a part of MEDICO',
                'Hi, Doctor. Thanks for become part of MEDICO.',
                settings.EMAIL_HOST_USER,
                [newdoctorusername],
                fail_silently=False,
            )
            #print(registereddoctors)

            pickle_out = open("doctors.pickle", "wb")
            pickledata = {'registereddoctors': registereddoctors}
            pickle.dump(pickledata, pickle_out)
            pickle_out.close()

            return redirect('doctordashboard')

        #print(newdoctorusername)
        #print(newdoctorpassword)
        #print(newdoctorconfirmpassword)


    return render(request, "doctorsignup.html")

def patientlogin(request):
    registeredpatients = {}

    if request.method == 'POST':

        patientusername = request.POST.get('patientusername')
        patientpassword = request.POST.get('patientpassword')

        try:
            pickle_in = open("patients.pickle", "rb")
            pickledata = pickle.load(pickle_in)
            registeredpatients = pickledata.get('registeredpatients')

        except:
            pass



        canPatientLogin = False

        #print(registeredpatients)
        #print(patientpassword)

        if len(registeredpatients)>0:
            for i in registeredpatients:
                #print(i)
```

```
                #print(registeredpatients.get(i).get("password"))
                if i == patientusername and
registeredpatients.get(i).get("password") == patientpassword:
                    print("Login Succesful.")
                    canPatientLogin = True
                    request.session["username"] = patientusername
                    break
                else:
                    print("please check your password")

            if canPatientLogin:
                return redirect('patientdashboard')
            else:
                print("please create account first.")

        else:
            print("please create account first")

    return render(request, "patientlogin.html")

def patientsignup(request):
    registeredpatients = {}

    if request.method == 'POST':

        newpatientusername = request.POST.get('newpatientusername')
        newpatientpassword = request.POST.get('newpatientpassword')
        newpatientconfirmpassword =
request.POST.get('newpatientconfirmpassword')

        try:
            pickle_in = open("patients.pickle", "rb")
            pickledata = pickle.load(pickle_in)
            registeredpatients = pickledata.get('registeredpatients')

        except:
            pass

        canCreateNewPatient = True
        if len(registeredpatients)>0:
            for i in registeredpatients:
                if i == newpatientusername:
                    print("patient with this username already exist.")
                    canCreateNewPatient = False

        if canCreateNewPatient:

            registeredpatients[newpatientusername] =
{"username":newpatientusername, "password":newpatientpassword}

            send_mail(
                'Thanks to be a part of MEDICO',
                'Hi there. Thanks for joining. Make an appoitment with our
professional doctors. Stay Healthy!',
                settings.EMAIL_HOST_USER,
                [newpatientusername],
                fail_silently=False,
```

```
            )

            print(registeredpatients)

            pickle_out = open("patients.pickle", "wb")
            pickledata = {'registeredpatients': registeredpatients}
            pickle.dump(pickledata, pickle_out)
            pickle_out.close()

            request.session["username"] = newpatientusername

            return redirect('patientdashboard')

        #print(newpatientusername)
        #print(newpatientpassword)
        #print(newpatientconfirmpassword)

    return render(request, "patientsignup.html")

def doctordashboard(request):

    if 'logout2' in request.POST:
        return redirect('/')


    d_name_list = []
    d_age_list = []
    d_address_list = []
    d_date_list = []
    d_time_list = []
    d_message_list = []
    d_status_list = []

    bookingdata = {}


    #print(username)
    try:

        pickle_in = open("booking.pickle", "rb")
        pickledata = pickle.load(pickle_in)
        bookingdata = pickledata.get('bookingdata')

    except:
        pass

    try:
        username = request.session["username"]
    except:
        pass

        #bookingdata[username] = {"username":username,
"doctorname":doctorname, "timeslot": timeslot, "status": "NA"}
    #print(bookingdata)


    doctorusername = request.session["doctorusername"]
```

```
    print(doctorusername)
    for i in bookingdata:
        if doctorusername == bookingdata.get(i).get("doctorname"):
            d_name_list.append(bookingdata.get(i).get("username"))
            d_age_list.append(bookingdata.get(i).get("age"))
            d_address_list.append(bookingdata.get(i).get("address"))
            d_date_list.append(bookingdata.get(i).get("date"))
            d_time_list.append(bookingdata.get(i).get("timeslot"))
            d_message_list.append(bookingdata.get(i).get("message"))
            d_status_list.append(bookingdata.get(i).get("status"))


    request_list =
list(zip(d_name_list,d_age_list,d_address_list,d_date_list,d_time_list,d_mess
age_list,d_status_list))
    #print(request_list)

    if request.method == 'POST':

        try:
            pickle_in = open("booking.pickle", "rb")
            pickledata = pickle.load(pickle_in)
            bookingdata = pickledata.get('bookingdata')

        except:
            pass

        approvepatientname = request.POST.get('approvepatient')
        #print(approvepatientname)

        if 'acceptbutton' in request.POST:
            send_mail(
                'Appointment Alert',
                'Hi there. Your appoitment with the doctor is Accepted.',
                settings.EMAIL_HOST_USER,
                [approvepatientname],
                fail_silently=False,
            )

            bookingdata.get(approvepatientname)["status"] = "Accepted"

        if 'rejectbutton' in request.POST:
            send_mail(
                'Appointment Alert',
                'Hi there. Your appoitment with the doctor is Rejected.',
                settings.EMAIL_HOST_USER,
                [approvepatientname],
                fail_silently=False,
            )

            bookingdata.get(approvepatientname)["status"] = "Rejected"


        new_status = bookingdata.get(approvepatientname).get('status')

        d_name_list.clear()
        d_age_list.clear()
```

```
        d_address_list.clear()
        d_date_list.clear()
        d_time_list.clear()
        d_message_list.clear()
        d_status_list.clear()


        for i in bookingdata:
            if doctorusername == bookingdata.get(i).get("doctorname"):
                d_name_list.append(bookingdata.get(i).get("username"))
                d_age_list.append(bookingdata.get(i).get("age"))
                d_address_list.append(bookingdata.get(i).get("address"))
                d_date_list.append(bookingdata.get(i).get("date"))
                d_time_list.append(bookingdata.get(i).get("timeslot"))
                d_message_list.append(bookingdata.get(i).get("message"))
                d_status_list.append(bookingdata.get(i).get("status"))


        request_list =
list(zip(d_name_list,d_age_list,d_address_list,d_date_list,d_time_list,d_mess
age_list,d_status_list))
        print(request_list)
        pickle_out = open("booking.pickle", "wb")
        pickledata = {'bookingdata': bookingdata}
        pickle.dump(pickledata, pickle_out)
        pickle_out.close()

        return render(request, "doctordashboard.html",
context={"request_list":request_list})

    return render(request, "doctordashboard.html",
context={"request_list":request_list})
    #return render(request, "doctordashboard.html")

def patientdashboard(request):

    bookingdata = {}
    doctors = [] #to return in select

    doctor_name_list = []
    time_slot_list = []
    status_list =[]

    #final_list = []

    if 'status' in request.POST:
        username = request.session["username"]
        try:
            pickle_in = open("booking.pickle", "rb")
            pickledata = pickle.load(pickle_in)
            bookingdata = pickledata.get('bookingdata')

        except:
            pass

        #bookingdata[username] = {"username":username,
"doctorname":doctorname, "timeslot": timeslot, "status": "NA"}
```

```
        for i in bookingdata:
            if i == username:

doctor_name_list.append(bookingdata.get(username).get("doctorname").split("@"
)[0])

time_slot_list.append(bookingdata.get(username).get("timeslot"))
                status_list.append(bookingdata.get(username).get("status"))

        final_list = list(zip(doctor_name_list,time_slot_list,status_list))

        return render(request, "viewbookingstatus.html",
context={"final_list":final_list})

    if 'logout' in request.POST:
        return redirect('/')

    try:
        pickle_in = open("doctors.pickle", "rb")
        pickledata = pickle.load(pickle_in)
        registereddoctors = pickledata.get('registereddoctors')

        for i in registereddoctors:
            doctors.append(i.split("@")[0])

    except:
        pass


    if request.method == 'POST':

        username = request.session["username"]

        name = request.POST.get('name')
        age = request.POST.get('age')
        address = request.POST.get('address')
        doctorname = request.POST.get('doctorname')
        # date = request.POST.get('date')
        # timeslot = request.POST.get('timeslot')
        # message = request.POST.get('message')

        #print("doctorname: "+doctorname)
        doctorname = doctorname+"@gmail.com"

        request.session["temp_booking_username"] = username
        request.session["temp_booking_name"] = name
        request.session["temp_booking_age"] = age
        request.session["temp_booking_address"] = address
        request.session["temp_booking_doctorname"] = doctorname



        return redirect("bookingpanel")

    return render(request, "patientdashboard.html",
context={'register_doctor_list':doctors})
```

```
def viewbookingstatus(request):

    #if 'back' in request.POST:
        #return redirect('/')
    return render(request, "viewbookingstatus.html")

def bookingpanel(request):
    bookingdata = {}
    doctor_name_list = []
    time_slot_list= []
    status_list = []

    doctorname = request.session["temp_booking_doctorname"]
    try:
        pickle_in = open("doctors.pickle", "rb")
        pickledata = pickle.load(pickle_in)
        registereddoctors = pickledata.get('registereddoctors')
        print(registereddoctors)
    except:
        pass

    all_timeslots_available = ["10 am to 11 am", "11 am to 12 am", "1 pm to 2
pm", "5 pm to 6 pm", "6 pm to 7 pm", "7 pm to 8 pm"]
    current_booked_slots =
registereddoctors.get(doctorname).get("booked_slots")

    current_available_slots = []

    for i in all_timeslots_available:
        if i not in current_booked_slots:
            current_available_slots.append(i)

    print(current_available_slots)

    if request.method == 'POST':



        username = request.session["temp_booking_username"]
        name = request.session["temp_booking_name"]
        age = request.session["temp_booking_age"]
        address = request.session["temp_booking_address"]
        doctorname = request.session["temp_booking_doctorname"]

        date = request.POST.get('date')
        timeslot = request.POST.get('timeslot')
        message = request.POST.get('message')

        print(timeslot)

        print(doctorname)
        doctor_booking_time_slot =
registereddoctors.get(doctorname).get("booked_slots")
        doctor_booking_time_slot.append(timeslot)
        registereddoctors.get(doctorname)["booked_slots"] =
doctor_booking_time_slot
```

```
        try:
            pickle_in_booking = open("booking.pickle", "rb")
            pickledata_booking = pickle.load(pickle_in_booking)
            bookingdata = pickledata_booking.get('bookingdata')

        except:
            pass

        bookingdata[username] = {"username":username,
"doctorname":doctorname, "timeslot": timeslot, "status": "NA",
        "age":age, "address":address, "date":date, "message":message}

        for i in bookingdata:
            if i == username:

doctor_name_list.append(bookingdata.get(username).get("doctorname"))

time_slot_list.append(bookingdata.get(username).get("timeslot"))
                status_list.append(bookingdata.get(username).get("status"))

        final_list = list(zip(doctor_name_list,time_slot_list,status_list))
        print(final_list)

        send_mail(
                'Appointment Alert',
                'Hi Doctor. Patient with the Unique ID : '+username+' wants
to make an appoitment with you. Kindly help!',
                settings.EMAIL_HOST_USER,
                [doctorname],
                fail_silently=False,
            )

        send_mail(
                'Appointment Alert',
                'Hi there. Your appoitment with the doctor is booked
successfull. Kindly wait for doctor response.',
                settings.EMAIL_HOST_USER,
                [username],
                fail_silently=False,
            )

        pickle_out_booking = open("booking.pickle", "wb")
        pickledata_booking = {'bookingdata': bookingdata}
        pickle.dump(pickledata_booking, pickle_out_booking)
        pickle_out_booking.close()

        pickle_out = open("doctors.pickle", "wb")
        pickledata = {'registereddoctors': registereddoctors}
        pickle.dump(pickledata, pickle_out)
        pickle_out.close()

        return render(request, "viewbookingstatus.html",
context={"final_list":final_list})
```

```
    return render(request,
"bookingpanel.html",context={"current_available_slots":current_available_slot
s})
```

2. HTML Detection Panel Code

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="utf-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.1.0/dist/css/bootstrap.min.
css" rel="stylesheet" integrity="sha384-
KyZXEAg3QhqLMpG8r+8fhAXLRk2vvoC2f3B09zVXn8CA5QIVfZOJ3BCsw2P0p/We"
crossorigin="anonymous">

    <title>MEDICO.KLU</title>
  {% load static %}
  </head>

  <nav class="navbar navbar-expand-lg navbar-light bg-light">
  <div class="container-fluid">
    <a class="navbar-brand" href="">MEDICO.KLU</a>
    <button class="navbar-toggler" type="button" data-bs-toggle="collapse"
data-bs-target="#navbarText" aria-controls="navbarText" aria-
expanded="false" aria-label="Toggle navigation">
      <span class="navbar-toggler-icon"></span>
    </button>
    <div class="collapse navbar-collapse" id="navbarText">
      <ul class="navbar-nav me-auto mb-2 mb-lg-0">
        <li class="nav-item">
          <a class="nav-link" aria-current="page" href="">Home</a>
        </li>
        <li class="nav-item">
          <a class="nav-link" href="about">About</a>
        </li>
        <li class="nav-item">
          <a class="nav-link" href="doctors">Doctors</a>
        </li>
        <li class="nav-item">
          <a class="nav-link" href="departments">Departments</a>
        </li>
        <li class="nav-item dropdown">
          <a class="nav-link dropdown-toggle" href="#"
id="navbarScrollingDropdown" role="button" data-bs-toggle="dropdown" aria-
expanded="false">
            Signup
          </a>
          <ul class="dropdown-menu" aria-
labelledby="navbarScrollingDropdown">
            <li><a class="dropdown-item"
href="doctorsignup">Doctor</a></li>
```

```
            <li><a class="dropdown-item"
href="patientsignup">Patient</a></li>
          </ul>
        </li>

        <li class="nav-item dropdown">
          <a class="nav-link dropdown-toggle" href="#"
id="navbarScrollingDropdown" role="button" data-bs-toggle="dropdown" aria-
expanded="false">
            Login
          </a>
          <ul class="dropdown-menu" aria-
labelledby="navbarScrollingDropdown">
            <li><a class="dropdown-item"
href="doctorlogin">Doctor</a></li>
            <li><a class="dropdown-item"
href="patientlogin">Patient</a></li>
          </ul>
        </li>
      </ul>
    </div>
  </div>
</nav>

  {% load static %}
  <body background="" style="background-size: cover;background-repeat: no-
repeat;background-attachment: fixed;">
    <script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.1.0/dist/js/bootstrap.bundle
.min.js" integrity="sha384-
U1DAWAznBHeqEIlVSCgzq+c9gqGAJn5c/t99JyeKa9xxaYpSvHU5awsuZVVFIhvj"
crossorigin="anonymous"></script>



    <div class="row">
      <img src="{% static 'myapp/dept-4.jpg' %}">
    </div>
    <br>
    <br>
    <div class="card" style="text-align: center;">
      <div class="card-body">
         <h1>We Care About Your Health</h1>
         <h5>Your health is our top priority with comprehensive,
affordable medical.</h5>
      </div>
    </div>



    <div class="row row-cols-1 row-cols-md-4 g-4" style="margin: 60px;">
      <div class="col">
        <div class="card">
          <img src="{% static 'myapp/1.png' %}" class="card-img-top"
alt="...">
          <div class="card-body">
            <h5 class="card-title">Qualified Doctors</h5>
```

```
          </div>
        </div>
      </div>
      <div class="col">
        <div class="card">
          <img src="{% static 'myapp/ambulance.png' %}" class="card-img-
top" alt="...">
          <div class="card-body">
            <h5 class="card-title">Emergency Care</h5>
          </div>
        </div>
      </div>
      <div class="col">
        <div class="card">
          <img src="{% static 'myapp/logo-dark.png' %}" class="card-img-
top" alt="...">
          <div class="card-body">
            <h5 class="card-title">Outdor Checkup</h5>
          </div>
        </div>
      </div>
      <div class="col">
        <div class="card">
          <img src="{% static 'myapp/24-hours.png' %}" class="card-img-
top" alt="...">
          <div class="card-body">
            <h5 class="card-title">24 Hour Service</h5>
          </div>
        </div>
      </div>
    </div>

  </body>
</html>
```

CHAPTER 5

TESTING

Testing Phase

Testing refers to test the software so it is also called software testing. **Software testing** is an investigation conducted to provide stakeholders with information about the quality of the software product or service under test.[1] Software testing can also provide an objective, independent view of the software to allow the business to appreciate and understand the risks of software implementation. Test techniques include the process of executing a program or application with the intent of finding software bugs (errors or other defects), and verifying that the software product is fit for use.

Software testing involves the execution of a software component or system component to evaluate one or more properties of interest. In general, these properties indicate the extent to which the component or system under test-

- meets the requirements that guided its design and development,
- responds correctly to all kinds of inputs,
- performs its functions within an acceptable time
- is sufficiently usable
- can be installed and run in its intended environments, and
- achieves the general result its stakeholders desire.

5.1 Unit Testing

In computer programming, unit testing is a software testing method by which individual units of source code, sets of one or more computer program modules together with associated control data, usage procedures, and operating procedures, are tested to determine whether they are fit for use.

Parameterized unit tests (PUTs) are tests that take parameters. Unlike traditional unit tests, which are usually closed methods, PUTs take any set of parameters. PUTs have been supported by Testing, JUnit and various .NET test frameworks.

Suitable parameters for the unit tests may be supplied manually or in some cases are automatically generated by the test framework. Testing tools like Quick Check exist to generate test inputs for PUTs.

The goal of unit testing is to isolate each part of the program and show that the individual parts are correct. A unit test provides a strict, written contract that the piece of code must satisfy. As a result, it affords several benefits.

We tested each and every unit (or module) and it was successfully executed.

All units are working properly and perform their operation perfectly as expected.

5.2 Integration Testing

Integration testing (sometimes called integration and testing, abbreviated I&T) is the phase in software testing in which individual software modules are combined and tested as a group. It occurs after unit testing and before validation testing.

Integration testing takes as its input modules that have been unit tested, groups them in larger aggregates, applies tests defined in an integration test plan to those aggregates, and delivers as its output the integrated system ready for system testing.

Some different types of integration testing are big-bang, mixed (sandwich), risky- hardest, top-down, and bottom-up. Other Integration Patterns are: collaboration integration, backbone integration, layer integration, client-server integration, distributed services integration and high-frequency integration.

In the big-bang approach, most of the developed modules are coupled together to form a complete software system or major part of the system and then used for integration testing. This method is very effective for saving time in the integration testing process.

However, if the test cases and their results are not recorded properly, the entire integration process will be more complicated and may prevent the testing team from achieving the goal of integration testing.

In our proposed system we tested all components by merge together and it passed all our criteria.

5.3 System Testing

System testing of software or hardware is testing conducted on a complete, integrated system to evaluate the system's compliance with its specified requirements. System testing falls within the scope of black-box testing, and as such, should require no knowledge of the inner design of the code or logic.

As a rule, system testing takes, as its input, all of the "integrated" software components that have passed integration testing and also the software system itself integrated with any applicable hardware system.

The purpose of integration testing is to detect any inconsistencies between the software units that are integrated together or between any of the assemblages and the hardware. System testing is a more limited type of testing; it seeks to detect defects both within the "inter-assemblages" and also within the system as a whole.

System testing is performed on the entire system in the context of a Functional Requirement Specification and/or a System Requirement Specification (SRS). System testing tests not only the design, but also the behaviour and even the believed expectations of the customer. It is also intended to test up to and beyond the bounds defined in the software/hardware requirements specification.

After performing system testing with our currency detection system we find that it follows all requirements and working as per requirement, it takes the input as

currency notes and after apply algorithm it give the proper result.

5.4 Functional Testing

Functional testing is a quality assurance (QA) process and a type of black-box testing that bases its test cases on the specifications of the software component under test.

Functions are tested by feeding them input and examining the output, and internal program structure is rarely considered (unlike white-box testing). Functional testing is conducted to evaluate the compliance of a system or component with specified functional requirements. Functional testing usually describes what the system does.

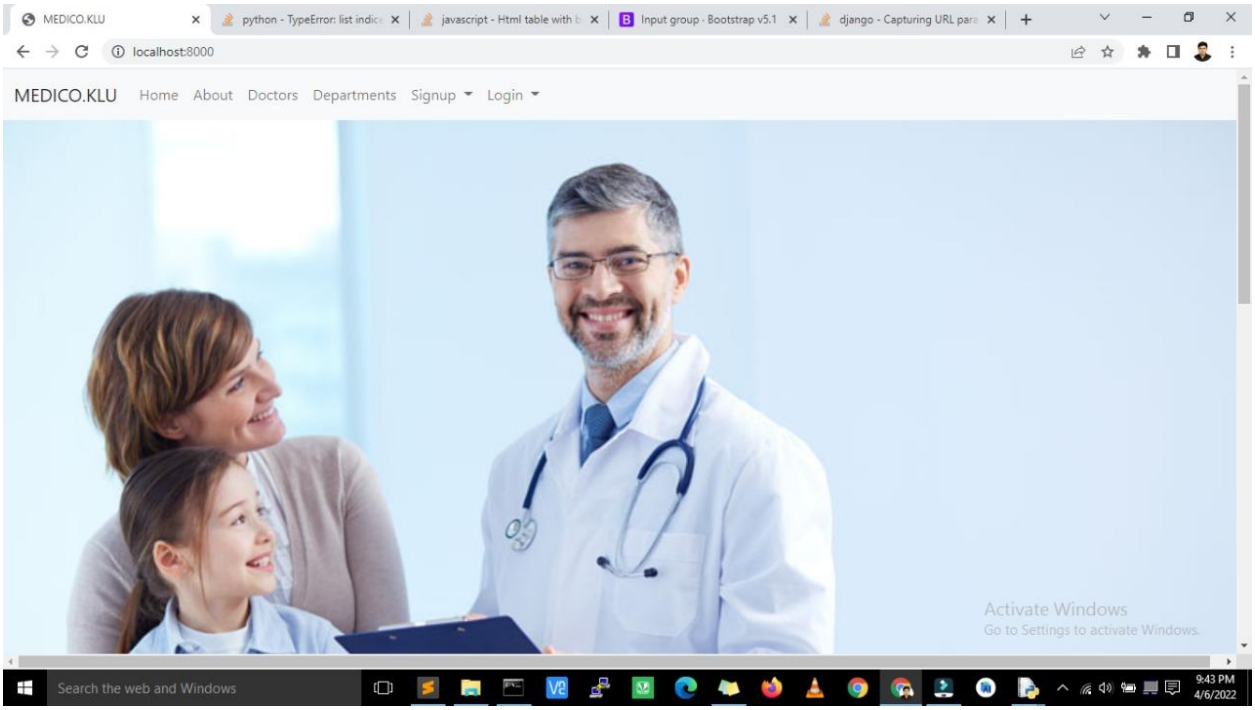
All the functionality of the appointment management system is working properly as expected.

Appointment booking functionality working perfectly and also send the acknowledgement mails where needed.

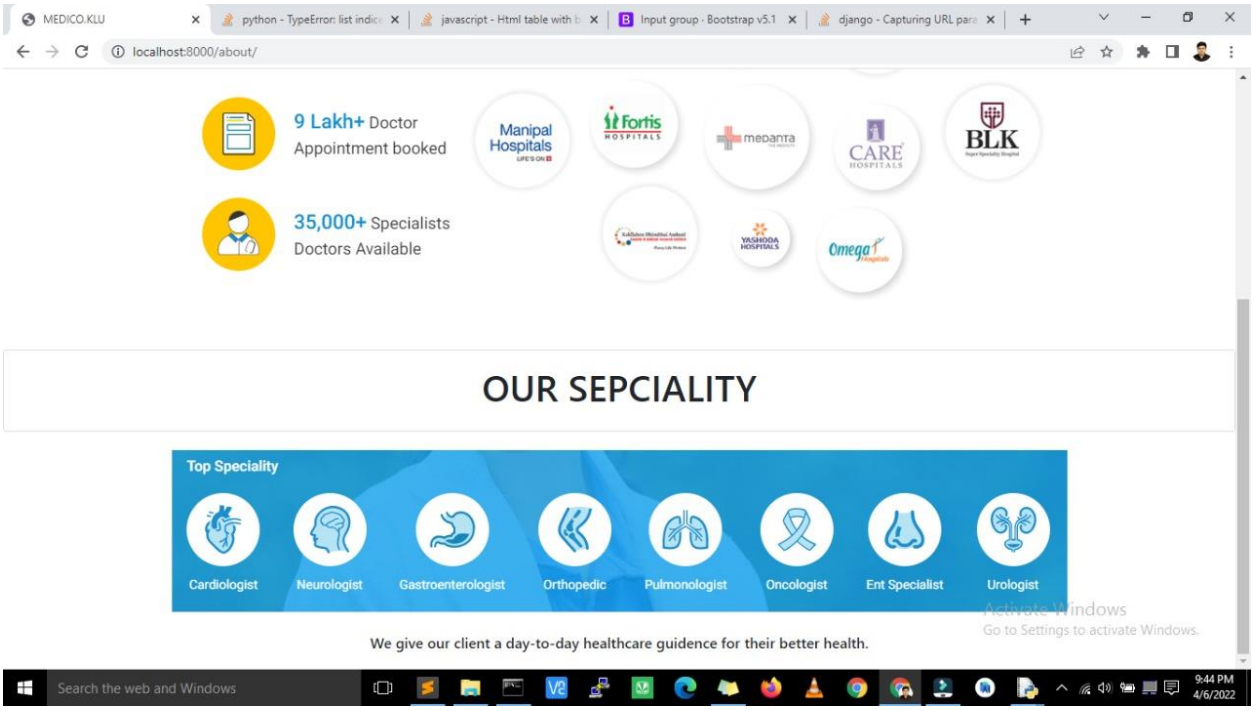
CHAPTER 6

SNAPSHOTS

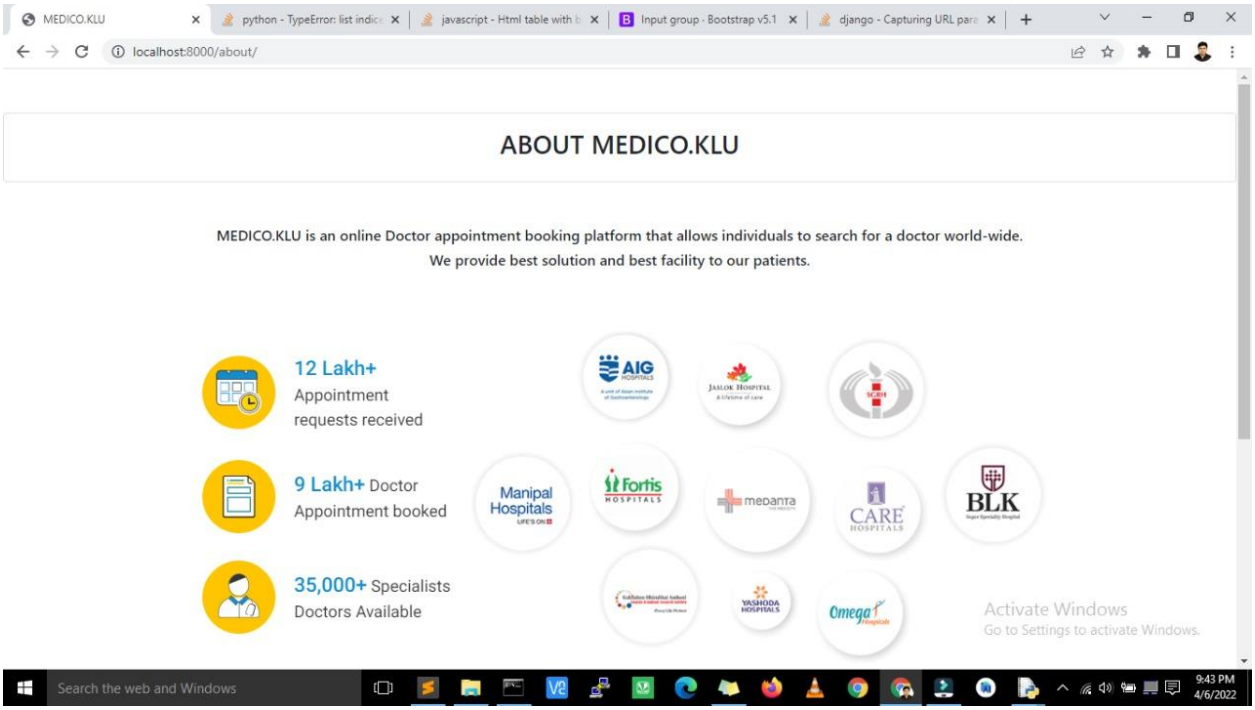
1.


MEDICO.KLU
python - TypeError: list indice
javascript - Html table with b
Input group · Bootstrap v5.1
django - Capturing URL para
localhost:8000
MEDICO.KLU
Home
About
Doctors
Departments
Signup
Login
Activate Windows
Go to Settings to activate Windows.
Search the web and Windows
9:43 PM
4/6/2022

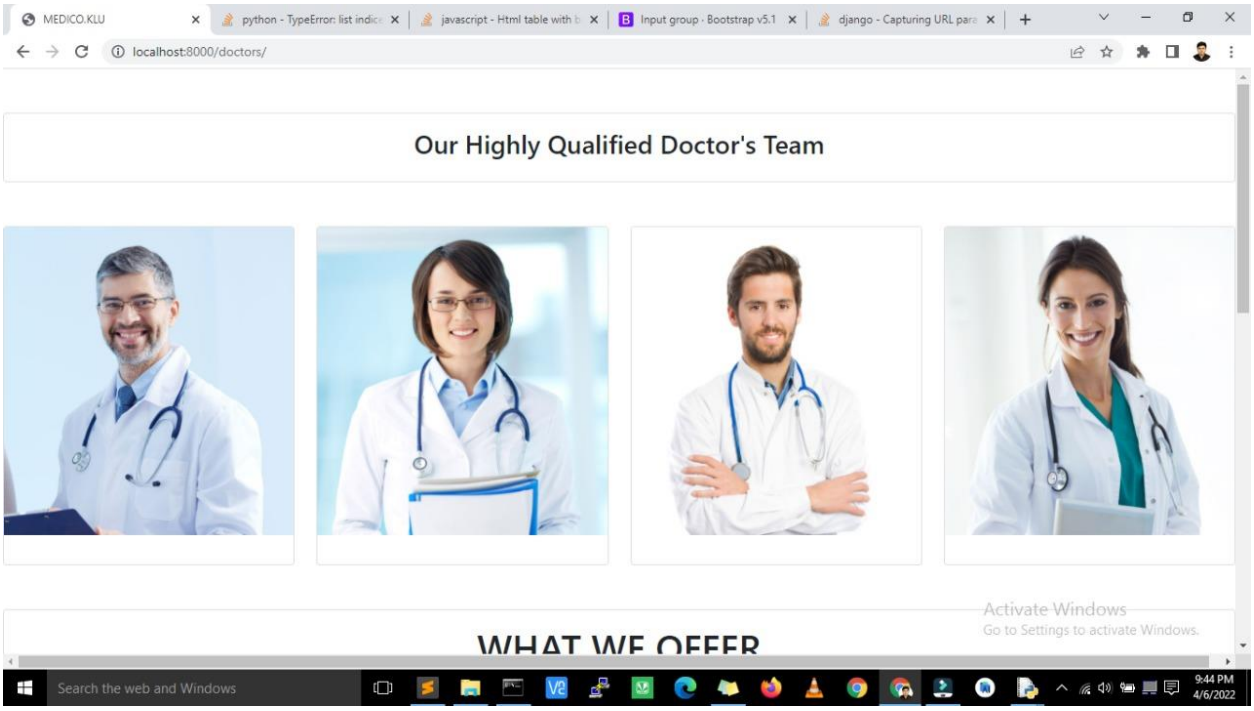
2.


MEDICO.KLU
python - TypeError: list indice
javascript - Html table with b
Input group · Bootstrap v5.1
django - Capturing URL para
localhost:8000/about/
9 Lakh+ Doctor
Appointment booked
35,000+ Specialists
Doctors Available
Manipal Hospitals
Fortis
medanta
CARE
BLK
YASHODA HOSPITALS
Omega
OUR SEPCIALITY
Top Speciality
Cardiologist
Neurologist
Gastroenterologist
Orthopedic
Pulmonologist
Oncologist
Ent Specialist
Urologist
We give our client a day-to-day healthcare guidence for their better health.
Activate Windows
Go to Settings to activate Windows.
Search the web and Windows
9:44 PM
4/6/2022

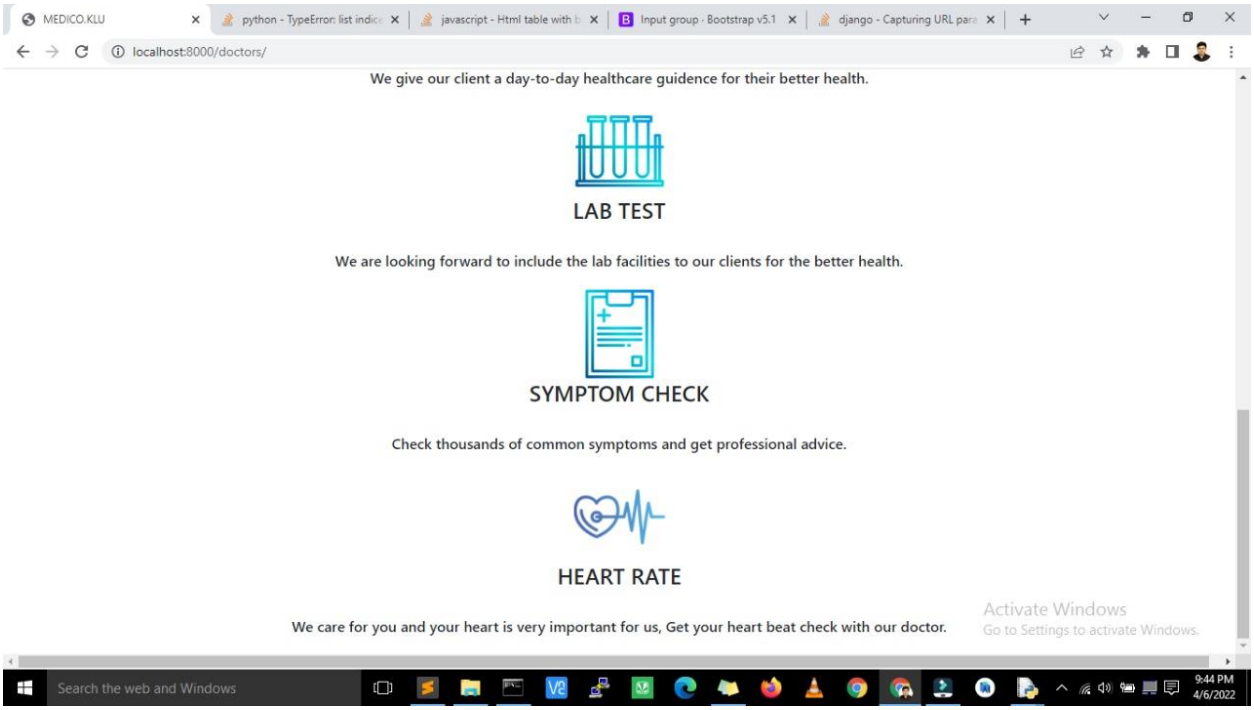
3.

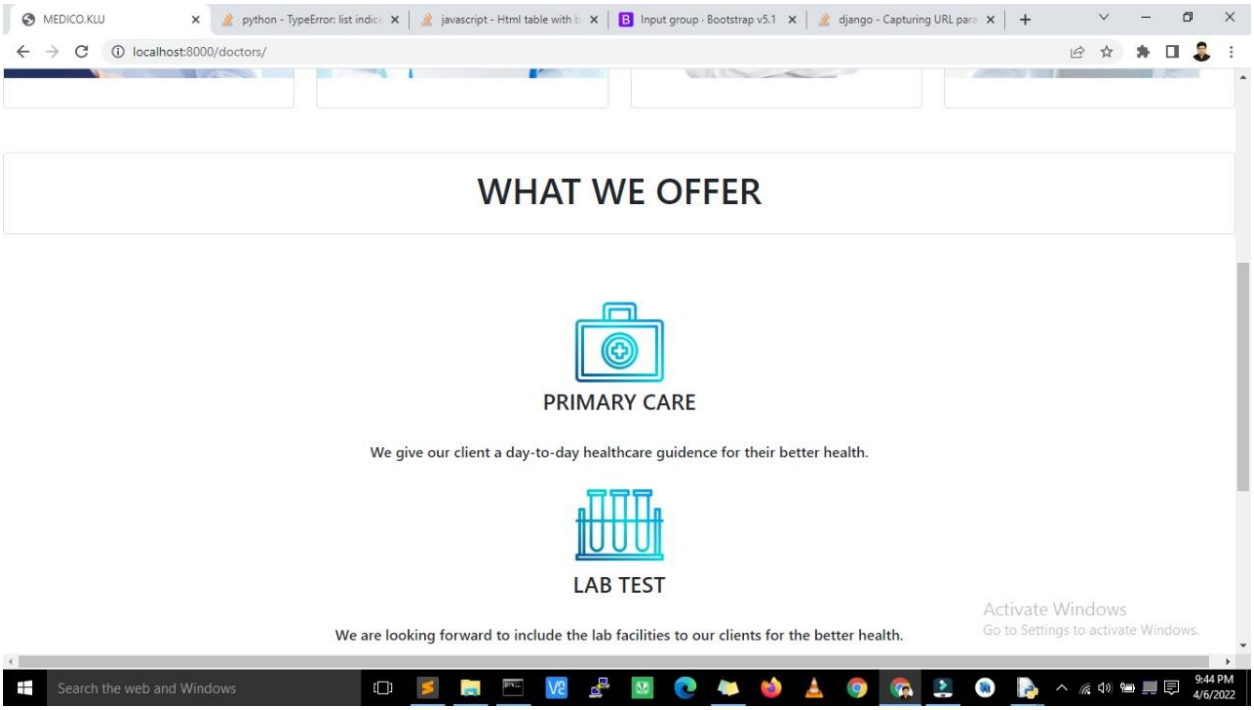

MEDICO.KLU
localhost:8000/about/
ABOUT MEDICO.KLU
MEDICO.KLU is an online Doctor appointment booking platform that allows individuals to search for a doctor world-wide.
We provide best solution and best facility to our patients.
12 Lakh+
Appointment
requests received
9 Lakh+ Doctor
Appointment booked
35,000+ Specialists
Doctors Available
AIG
Manipal Hospitals
Fortis
medanta
CARE
BLK
Omega
Activate Windows
Go to Settings to activate Windows.
Search the web and Windows
9:43 PM
4/6/2022

4.

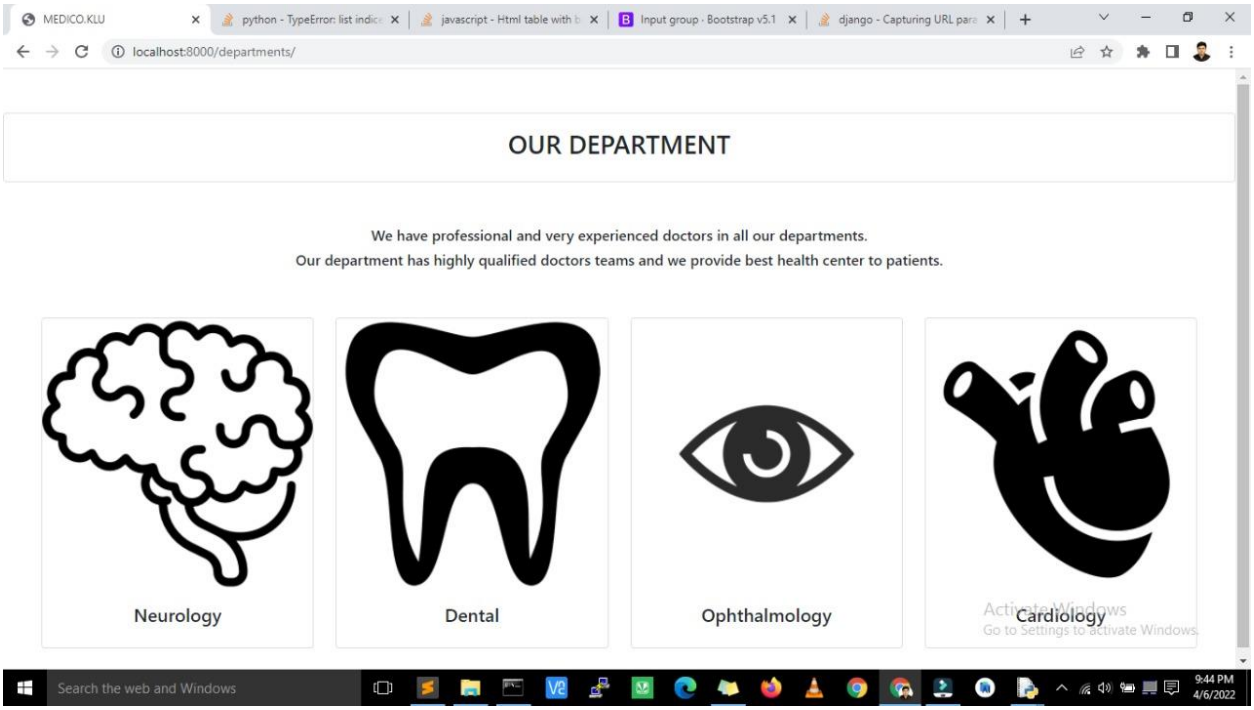

MEDICO.KLU
localhost:8000/doctors/
Our Highly Qualified Doctor's Team
WHAT WE OFFER
Activate Windows
Go to Settings to activate Windows.
Search the web and Windows
9:44 PM
4/6/2022

5.

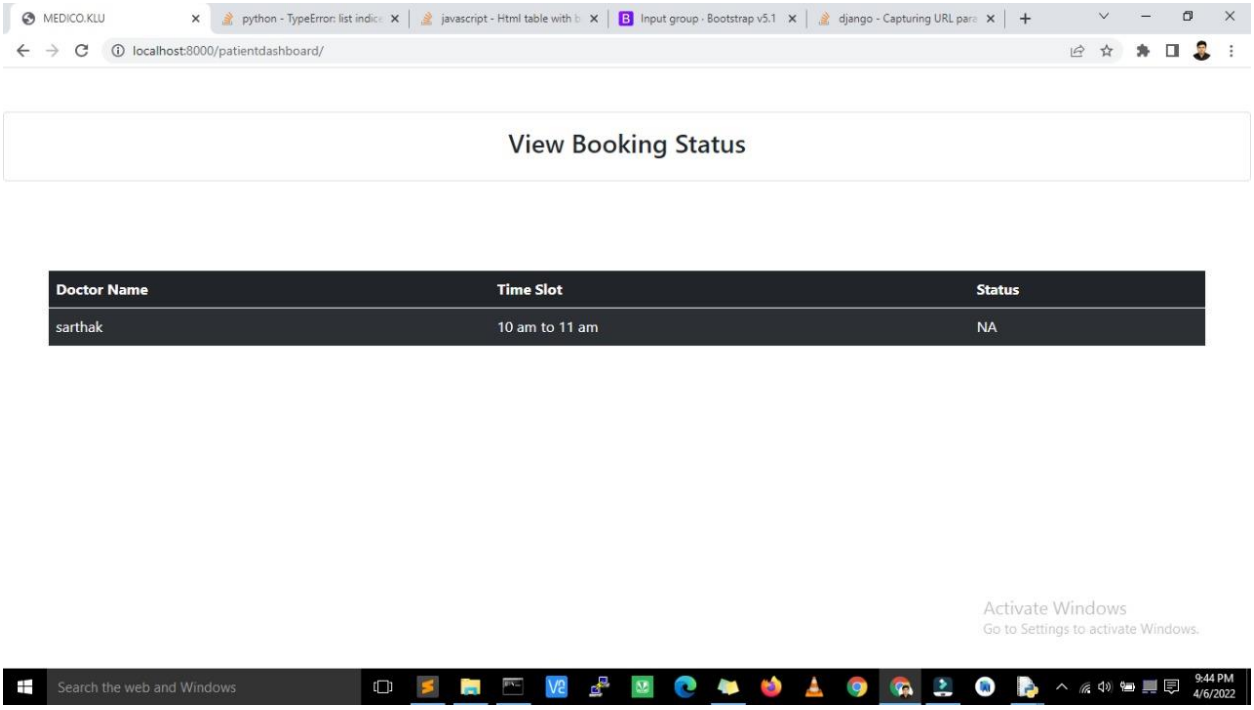

MEDICO.KLU
localhost:8000/doctors/
We give our client a day-to-day healthcare guidence for their better health.
LAB TEST
We are looking forward to include the lab facilities to our clients for the better health.
SYMPTOM CHECK
Check thousands of common symptoms and get professional advice.
HEART RATE
We care for you and your heart is very important for us, Get your heart beat check with our doctor.
Activate Windows
Go to Settings to activate Windows.
Search the web and Windows
9:44 PM
4/6/2022


MEDICO.KLU
localhost:8000/doctors/
WHAT WE OFFER
PRIMARY CARE
We give our client a day-to-day healthcare guidence for their better health.
LAB TEST
We are looking forward to include the lab facilities to our clients for the better health.
Activate Windows
Go to Settings to activate Windows.
Search the web and Windows
9:44 PM
4/6/2022

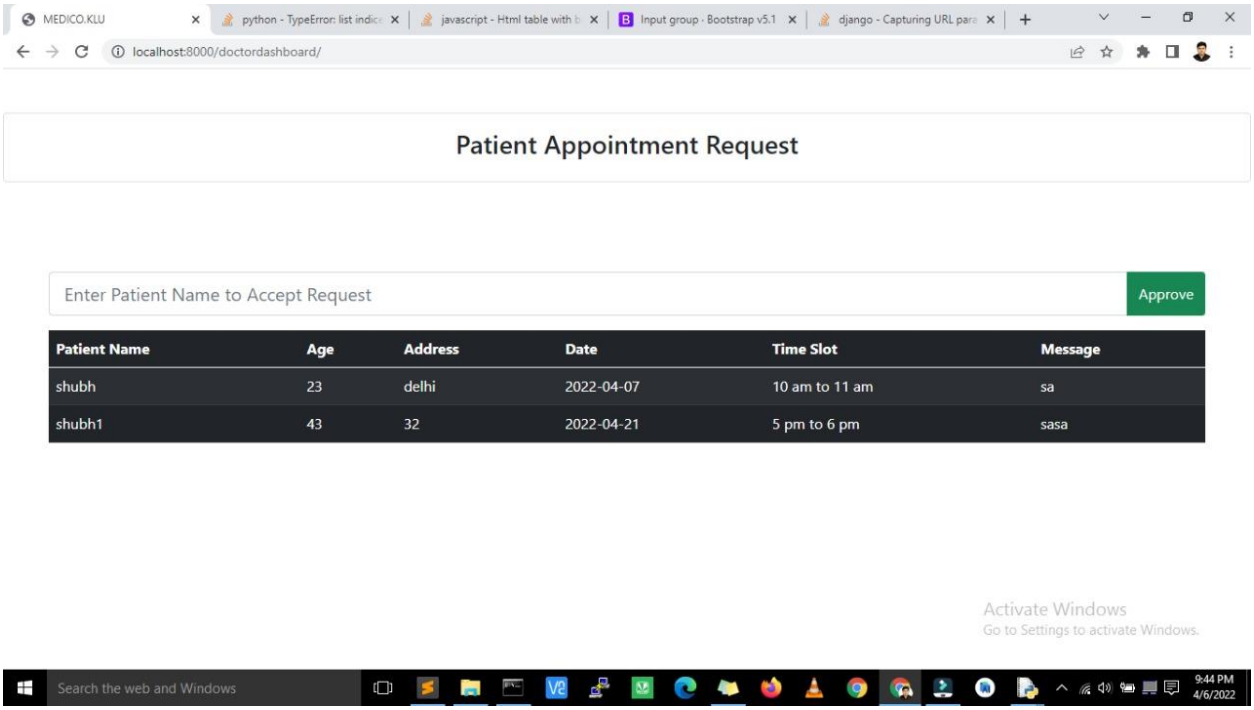
6.


MEDICO.KLU
localhost:8000/departments/
OUR DEPARTMENT
We have professional and very experienced doctors in all our departments.
Our department has highly qualified doctors teams and we provide best health center to patients.
Neurology
Dental
Ophthalmology
Cardiology

7.


MEDICO.KLU
localhost:8000/patientdashboard/
View Booking Status
Doctor Name
Time Slot
Status
sarthak
10 am to 11 am
NA

8.


MEDICO.KLU
python - TypeError: list indice
javascript - Html table with b
Input group · Bootstrap v5.1
django - Capturing URL para
localhost:8000/doctordashboard/
Patient Appointment Request
Enter Patient Name to Accept Request
Approve
Patient Name | Age | Address | Date | Time Slot | Message
shubh | 23 | delhi | 2022-04-07 | 10 am to 11 am | sa
shubh1 | 43 | 32 | 2022-04-21 | 5 pm to 6 pm | sasa
Activate Windows
Go to Settings to activate Windows.
Search the web and Windows
9:44 PM
4/6/2022

CHAPTER 7

FUTURE ENHANCEMENT

Future Enhancement

In Future we will add some more features to our system like-

1. In future we can add the video call feature so that patient can consult with doctor online.

2. In future we will give mobile support also using our android application.

3. In future we will make this application live so that anyone can access this application easily.

4. In future we will add features like SMS alerts.

5. In future we will add the recommendation system also so that doctor can be recommended to the patient based on their symptoms.

CONCLUSION

Doctors and patients' appointment web application is a very exciting topic to work on. After going through the work or while creating the application, we faced so many challenging tasks. Day by day the healthcare system has become an important part of our society. So we have to decide to build this web application. We researched so many systems that showed us the direction how to develop our web application. We also interact with some of the people about what type of problem they are facing while they are going to take the appointment to the doctor. They were very happy to take this web application as it gives them some relief in the modern age. Despite everything we achieved, we faced many challenges while working on this whole web application. After all it’s an online web-based appointment web application so in real life both doctor and patient need to follow all the rules otherwise its goal will be failed in future. Online web application is always a changeable system. It develops day by day, getting better and better and easier for people. This could be a revolutionary web application that may help bonding between doctor and patient. We believe we can make this system more advanced in future. Advance features and User interface will be updated in future. Our system is already user friendly but we will try to make this system more user friendly in future.

REFERENCES

BOOKS:

- Ian Somerville ‘Software engineering’
- Rajeev mall ‘Software engineering’

ONLINE REFERENCE:

1. www.google.co.in
2. https://stackoverflow.com/
3. https://docs.djangoproject.com/en/3.2/
4. https://www.w3schools.com/